Diseño, Correctitud y Análisis de Complejidad de un Árbol Binario de Búsqueda Implícito mediante Treaps con Propagación Perezosa

Thiago Rivera, Emanuel Bañon, Tamy Flores, Jesús Niño
Ciencia de la Computación, Universidad de Ingeniería y Tecnología (UTEC), Lima, Perú
CS3014 – Estructuras de Datos Avanzadas

***Resumen*—El mantenimiento eficiente de secuencias dinámicas sujetas tanto a modificaciones estructurales por posición como a actualizaciones y consultas sobre intervalos constituye un problema fundamental en el diseño de estructuras de datos avanzadas. Mientras que los arreglos contiguos ofrecen acceso posicional en $\mathcal{O}(1)$ pero requieren $\mathcal{O}(n)$ desplazamientos ante inserciones o eliminaciones, las listas enlazadas invierten este compromiso sin lograr eficiencia simultánea. En este reporte se presenta el diseño e implementación de un Árbol Binario de Búsqueda Implícito basado en un *Treap Implícito* (*Implicit Treap*) con propagación perezosa (*lazy propagation*) y gestión de memoria mediante un *pool* estático contiguo. La estructura soporta seis operaciones sobre una secuencia de hasta $n, q \leq 5 \cdot 10^5$ elementos: suma en rango (`A`), reversión de subarreglo (`R`), rotación circular derecha en rango (`O`), inserción posicional (`I`), eliminación posicional (`E`) y consulta de mínimo en rango (`M`). Se demuestra formalmente la correctitud de las primitivas `split` y `merge`, la conmutatividad de las etiquetas diferidas y la cota de tiempo esperado $\mathcal{O}(\log n)$ por operación con complejidad espacial lineal $\mathcal{O}(n+q)$.**

***Index Terms*—BST Implícito, Treap, Propagación Perezosa, Árbol Cartesiano Aleatorizado, Consultas de Rango Dinámicas.**

I. Introducción

Las estructuras de datos lineales clásicas presentan una dicotomía inherente cuando se requiere manipular secuencias cuyo contenido y topología cambian a lo largo del tiempo. Un arreglo estático o dinámico mantiene los elementos en celdas físicamente contiguas, garantizando una localidad espacial óptima y acceso directo a la k-ésima posición en tiempo $\mathcal{O}(1)$; sin embargo, insertar o eliminar un elemento en una posición arbitraria obliga a desplazar $\Theta(n)$ elementos en el peor caso. Incluso al relajar la contigüidad estricta mediante estructuras como un *Packed-Memory Array* para el problema de *Ordered File Maintenance*, las actualizaciones puntuales requieren $\mathcal{O}(\log^2 n)$ movimientos amortizados y no permiten revertir ni rotar subarreglos arbitrarios en tiempo sublineal. En el extremo opuesto, una lista doblemente enlazada permite insertar o suprimir un nodo conocido en $\mathcal{O}(1)$, pero carece de acceso aleatorio, degradando la búsqueda posicional a $\mathcal{O}(n)$.

Los Árboles Binarios de Búsqueda (BST) balanceados superan esta barrera logrando tiempos $\mathcal{O}(\log n)$ tanto para búsquedas como para actualizaciones. No obstante, en un BST estándar la ubicación de cada nodo depende de una llave explícita fija, lo cual resulta inadecuado cuando el problema exige indexar los datos por su posición actual $i \in \{1, \ldots, |a|\}$ y ejecutar transformaciones estructurales masivas que alteran los índices de múltiples elementos simultáneamente.

Un **BST Implícito** resuelve esta limitación eliminando el almacenamiento de llaves explícitas: la llave de cada nodo queda definida implícitamente por su posición dentro del recorrido en orden (*inorder traversal*) del árbol, calculada en tiempo real a partir del tamaño de los subárboles. En este trabajo se aborda la implementación de un BST implícito capaz de mantener una secuencia inicial $a_1, a_2, \ldots, a_n$ ($1 \leq n \leq 5 \cdot 10^5$, $|a_i| \leq 10^9$) y procesar en línea $q \leq 5 \cdot 10^5$ consultas de seis tipos:

- `A l r x`: Sumar x a cada a_i con $i \in [l, r]$ ($a_i \leftarrow a_i + x$).
- `R l r`: Invertir el orden de los elementos en el rango $[l, r]$.
- `O l r k`: Rotar circularmente hacia la derecha el subarreglo $a[l \ldots r]$ exactamente k posiciones.
- `I k x`: Insertar el valor x inmediatamente a la derecha de a_k (o al inicio si $k = 0$).
- `E k`: Eliminar el elemento ubicado en la posición k.
- `M l r`: Calcular el mínimo del rango, $\min_{i=l}^{r}\{a_i\}$.

II. Diseño de Implementación

II-A. Elección de la Estructura: Treap Implícito

Para materializar el BST implícito bajo límites estrictos (6.0 s de tiempo y 512 MB de memoria para $5 \cdot 10^5$ consultas), existen dos alternativas principales capaces de separar y unir árboles eficientemente: los *Splay Trees* (autoajustables con costo amortizado $\mathcal{O}(\log n)$) y los *Treaps Implícitos* (con costo esperado $\mathcal{O}(\log n)$). Se seleccionó el **Treap Implícito** debido a tres ventajas prácticas decisivas:

1. **Ausencia de punteros al padre:** A diferencia de un *Splay Tree*, que requiere mantener enlaces ascendentes y ejecutar rotaciones dobles (*Zig-Zig* y *Zig-Zag*), el Treap opera de arriba hacia abajo (*top-down*) mediante dos rutinas recursivas simples: `split` y `merge`.
2. **Menor factor constante con propagación perezosa:** En un Treap, las etiquetas diferidas se empujan una sola vez por nivel al descender en `split` o `merge`, evitando múltiples pasadas de actualización de invariantes.
3. **Resistencia a patrones adversarios:** Al asignar prioridades pseudoaleatorias independientes mediante el generador `std::mt19937`, la estructura del árbol es estadísticamente independiente de la secuencia de entrada, garantizando altura logarítmica con alta probabilidad.

II-B. Representación del Nodo y Tipos de Datos

Cada elemento de la secuencia se modela mediante una estructura `Node` que consolida los atributos de valor, información agregada del subárbol, etiquetas perezosas y topología del árbol:

Cuadro I
ESTRUCTURA DE DATOS DE UN NODO EN EL TREAP IMPLÍCITO

Campo	Tipo C++	Descripción y Rol Algorítmico
`val`	`long long`	Valor escalar actual del elemento a_i.
`mn`	`long long`	Mínimo agregado en todo el subárbol T_u.
`lazy_add`	`long long`	Incremento diferido pendiente para los hijos.
`prio`	`uint32_t`	Prioridad aleatoria uniforme (*Max-Heap*).
`sz`	`int`	Cardinalidad del subárbol (llave implícita).
`l, r`	`int`	Índices hacia el hijo izquierdo y derecho.
`rev`	`bool`	Bandera diferida de inversión de rango.

Prevención de desbordamiento aritmético: Dado que los valores iniciales satisfacen $|a_i| \leq 10^9$ y pueden ejecutarse hasta $q = 5 \cdot 10^5$ operaciones `A l r x` con $|x| \leq 10^9$, el valor absoluto de un elemento puede alcanzar:

$$\max |a_i| \leq 10^9 + q \cdot 10^9 = 5.001 \times 10^{14} \tag{1}$$

Esta magnitud supera ampliamente el límite de un entero con signo de 32 bits ($2^{31} - 1 \approx 2.14 \times 10^9$). Por ello, los campos `val`, `mn` y `lazy_add` se declaran como enteros de 64 bits (`long long`), cuyo rango máximo ($\approx 9.22 \times 10^{18}$) previene cualquier desbordamiento.

II-C. Gestión de Memoria: Pool Estático y Nodo Centinela

El uso reiterado de asignaciones dinámicas en el *heap* del sistema (`new` y `delete`) introduce sobrecarga temporal y fragmentación de memoria, degradando el rendimiento por fallos de caché (*cache misses*). Para maximizar la localidad espacial, se preasigna un arreglo global estático `tree[MAX_NODES]` con capacidad MAX_NODES $= 1\,000\,005 \geq n+q+5$, complementado con una pila de reciclaje `free_list[MAX_NODES]`:

- **Nodo centinela nulo (`u = 0`):** El índice 0 representa el puntero nulo. Se inicializa con $\text{sz}[0] = 0$ y $\text{mn}[0] = +\infty$ (LLONG_MAX/4). Esto permite consultar tree$[u.l]$.sz y tree$[u.l]$.mn directamente sin ramificaciones condicionales.
- **Asignación y reciclaje en $\mathcal{O}(1)$:** Cuando se ejecuta `E k`, el índice del nodo eliminado se apila en `free_list`. Cuando se ejecuta `I k x`, `new_node(x)` extrae un índice reciclado si la pila no está vacía o incrementa el contador `node_cnt` en caso contrario.

II-D. Primitivas de Mantenimiento y Propagación Perezosa

El mantenimiento de la información de los subárboles se apoya en cuatro funciones auxiliares de tiempo constante $\mathcal{O}(1)$:

1. `apply_add(u, x)`: Si $u \neq 0$, actualiza inmediatamente el nodo sumando x a val$[u]$, mn$[u]$ y lazy_add$[u]$.
2. `apply_rev(u)`: Si $u \neq 0$, invierte el estado de la bandera booleana (rev$[u] \leftarrow$ rev$[u] \oplus 1$) e intercambia inmediatamente los índices de sus hijos (swap$(u.l, u.r)$).
3. `push(u)`: Antes de descender hacia los hijos de u, si rev$[u]$ es verdadero, invoca `apply_rev(u.l)` y `apply_rev(u.r)`, y restablece rev$[u] \leftarrow$ false. Análogamente, si lazy_add$[u] \neq 0$, invoca `apply_add` sobre ambos hijos y restablece lazy_add$[u] \leftarrow 0$.
4. `pull(u)`: Tras modificar los hijos de u, recalcula los agregados del subárbol a partir de la información ya actualizada de sus hijos:

$$\text{sz}[u] = 1 + \text{sz}[u.l] + \text{sz}[u.r] \tag{2}$$

$$\text{mn}[u] = \min\{\text{val}[u], \text{mn}[u.l], \text{mn}[u.r]\} \tag{3}$$

Sobre estas rutinas se construyen las dos operaciones estructurales nucleares:

- `split(u, k, L, R)`: Divide el Treap enraizado en u en dos árboles L y R, de modo que L contiene los primeros k elementos del recorrido *inorder* y R contiene los $\text{sz}[u] - k$ restantes. Tras ejecutar `push(u)`, se compara k con $s_L = \text{sz}[u.l]$. Si $s_L \geq k$, el corte ocurre dentro del subárbol izquierdo: se llama recursivamente a `split(u.l, k, L, u.l)` y se asigna $R \leftarrow u$. Si $s_L < k$, el subárbol izquierdo y la raíz u pertenecen a L (aportando $s_L + 1$ nodos), por lo que se llama a `split(u.r, k - s_L - 1, u.r, R)` y se asigna $L \leftarrow u$. Al finalizar, se ejecuta `pull(u)`.
- `merge(L, R)`: Une dos Treaps L y R bajo la precondición de que toda posición implícita de L precede a las de R. Si alguno es 0, retorna el otro. De lo contrario, compara prio$[L]$ con prio$[R]$. Si prio$[L]$ > prio$[R]$, ejecuta `push(L)`, asigna $L.r \leftarrow \text{merge}(L.r, R)$, ejecuta `pull(L)` y retorna L. El caso simétrico procede sobre R.

II-E. Reducción de las Operaciones del Problema

Para operar sobre cualquier subarreglo $a[l \dots r]$ ($1 \leq l \leq r \leq |a|$), la secuencia se descompone en tres árboles disjuntos $T_1 = a[1 \dots l-1]$, $T_2 = a[l \dots r]$ y $T_3 = a[r+1 \dots |a|]$ mediante dos llamadas a `split`:

$$\texttt{split}(\text{root}, r, T_{12}, T_3); \quad \texttt{split}(T_{12}, l-1, T_1, T_2); \tag{4}$$

Con el rango aislado en la raíz T_2, cada consulta se resuelve como sigue:

- **Adición (`A l r x`):** Se aplica $\texttt{apply_add}(T_2, x)$ en $\mathcal{O}(1)$ y se recompone root $\leftarrow \texttt{merge}(\texttt{merge}(T_1, T_2), T_3)$.
- **Reversión (`R l r`):** Se aplica $\texttt{apply_rev}(T_2)$ en $\mathcal{O}(1)$ y se recompone root $\leftarrow \texttt{merge}(\texttt{merge}(T_1, T_2), T_3)$.
- **Rotación Derecha (`O l r k`):** Sea $\text{len} = r - l + 1$ y $k' = k \bmod \text{len}$. Si $k' > 0$, se aísla T_2 y se divide en $T_{2,L}$ de tamaño $\text{len} - k'$ y $T_{2,R}$ de tamaño k' mediante $\texttt{split}(T_2, \text{len} - k', T_{2,L}, T_{2,R})$. Luego se concatenan $T_2 \leftarrow \texttt{merge}(T_{2,R}, T_{2,L})$ y se restituye la raíz global.

- **Inserción (`I k x`):** Se ejecuta $\texttt{split}(\text{root}, k, T_1, T_2)$, separando los primeros k elementos, y se une root $\leftarrow \texttt{merge}(\texttt{merge}(T_1, \texttt{new_node}(x)), T_2)$.
- **Eliminación (`E k`):** Se aísla el rango unitario $[k, k]$ en T_2, se recicla el nodo con $\texttt{free_node}(T_2)$ y se actualiza root $\leftarrow \texttt{merge}(T_1, T_3)$.
- **Mínimo (`M l r`):** Se aísla $[l, r]$ en T_2, se lee la respuesta ans $= \text{mn}[T_2]$ en $\mathcal{O}(1)$ y se recompone la raíz.

III. Correctitud Algorítmica

III-A. Invariante de Llave Implícita y Operaciones Split/Merge

Lema 1 (Correctitud de `split`): *Sea u la raíz de un Treap válido que representa la secuencia inorder $S(u) = \langle e_1, \ldots, e_m \rangle$ y sea $0 \le k \le m$. La llamada `split(u, k, L, R)` produce dos Treaps válidos L y R tales que $S(L) = \langle e_1, \ldots, e_k \rangle$ y $S(R) = \langle e_{k+1}, \ldots, e_m \rangle$.*

Demostración (Por inducción sobre $m = sz[u]$):

- *Caso base ($m = 0$):* $u = 0$, la función asigna $L = R = 0$, cumpliendo trivialmente $S(L) = S(R) = \emptyset$.
- *Paso inductivo ($m > 0$):* Por definición del recorrido *inorder*, $S(u) = S(u.l) \parallel \langle u \rangle \parallel S(u.r)$, donde $|S(u.l)| = s_L = \text{sz}[u.l]$ y el nodo u ocupa la posición $s_L + 1$. Tras invocar `push(u)`, los subárboles $u.l$ y $u.r$ tienen sus valores y orientaciones actualizados.
  1. Si $s_L \ge k$, los primeros k elementos de $S(u)$ están contenidos en $S(u.l)$. Por hipótesis inductiva sobre $u.l$ (de tamaño $s_L < m$), `split(u.l, k, L, u.l')` deja en L los primeros k elementos y en $u.l'$ los elementos $\langle e_{k+1}, \ldots, e_{s_L} \rangle$. Al asignar $R \leftarrow u$, la secuencia de R es $S(u.l') \parallel \langle e_{s_L+1} \rangle \parallel S(u.r) = \langle e_{k+1}, \ldots, e_m \rangle$.
  2. Si $s_L < k$, todo $S(u.l)$ y la raíz u forman los primeros $s_L + 1 \le k$ elementos de $S(L)$. Faltan extraer $k' = k - s_L - 1$ elementos de $S(u.r)$. Por hipótesis inductiva sobre $u.r$, `split(u.r, k', u.r', R)` asigna a $u.r'$ los elementos $\langle e_{s_L+2}, \ldots, e_k \rangle$ y a R el sufijo $\langle e_{k+1}, \ldots, e_m \rangle$. Al fijar $L \leftarrow u$, se obtiene $S(L) = S(u.l) \parallel \langle e_{s_L+1} \rangle \parallel S(u.r') = \langle e_1, \ldots, e_k \rangle$.

  Dado que $u.l'$ (o $u.r'$) es un subconjunto de los descendientes originales de u, sus prioridades siguen siendo menores que prio$[u]$, preservando el invariante de *Max-Heap*. ■

Lema 2 (Correctitud de `merge`): *Dados dos Treaps válidos L y R, `merge(L, R)` retorna un Treap válido T tal que $S(T) = S(L) \parallel S(R)$.*

Demostración: Por inducción sobre $|S(L)|+|S(R)|$. El caso base ($L = 0$ o $R = 0$) es inmediato. En el paso inductivo, la raíz de T debe ser el nodo de mayor prioridad entre L y R para respetar la propiedad de *Max-Heap*. Si prio$[L]$ > prio$[R]$, L es la raíz; como todo elemento de $S(R)$ debe ubicarse después de $S(L)$, R debe fusionarse con el subárbol derecho de L. Por hipótesis inductiva, $S(\texttt{merge}(L.r, R)) = S(L.r) \parallel S(R)$, por lo que $S(T) = S(L.l) \parallel \langle L \rangle \parallel S(L.r) \parallel S(R) = S(L) \parallel S(R)$. El caso prio$[R] \ge$ prio$[L]$ es simétrico. ■

III-B. Conmutatividad de las Etiquetas Perezosas

Teorema 1 (Conmutatividad de Suma y Reversión): *Para toda secuencia $S = \langle x_1, x_2, \ldots, x_m \rangle$, el operador de adición escalar $\mathcal{A}_\Delta(S) = \langle x_1 + \Delta, \ldots, x_m + \Delta \rangle$ y el operador de inversión $\mathcal{R}(S) = \langle x_m, \ldots, x_1 \rangle$ conmutan y preservan el invariante del mínimo.*

Demostración: Evaluando ambas composiciones sobre el i-ésimo término ($1 \le i \le m$):

$$\begin{aligned} \big[\mathcal{A}_\Delta(\mathcal{R}(S))\big]_i &= [\mathcal{R}(S)]_i + \Delta = x_{m-i+1} + \Delta \\ &= \big[\mathcal{R}(\mathcal{A}_\Delta(S))\big]_i \end{aligned} \tag{5}$$

Por tanto, $\mathcal{A}_\Delta \circ \mathcal{R} = \mathcal{R} \circ \mathcal{A}_\Delta$. Además, como $\mathcal{R}$ es una biyección (permutación de índices) sobre el multiconjunto de valores de S, el mínimo es invariante bajo inversión ($\min \mathcal{R}(S) = \min S$) y distributivo respecto a la suma ($\min \mathcal{A}_\Delta(S) = (\min S) + \Delta$). En consecuencia, aplicar las etiquetas pendientes en `push(u)` en cualquier orden fijo (primero `rev` y luego `lazy_add`) es exactamente correcto sin importar el orden cronológico en que llegaron las consultas `A` y `R`. ■

III-C. Correctitud de la Rotación Circular en Rango

En la operación `O l r k`, sea $B[1 \ldots \text{len}]$ el subarreglo correspondiente a $a[l \ldots r]$ con len $= r - l + 1$. Cada rotación unitaria hacia la derecha traslada el último elemento $B[\text{len}]$ al inicio y desplaza el resto una posición a la derecha. Aplicar esta transformación k veces ($0 \le k <$ len) ubica en las primeras k posiciones a los últimos k elementos en su mismo orden relativo:

$$\text{Rotate}_k(B) = B[\text{len} - k + 1 \ldots \text{len}] \parallel B[1 \ldots \text{len} - k] \tag{6}$$

Al ejecutar $\texttt{split}(T_2, \text{len} - k, T_{2,L}, T_{2,R})$, el Lema 1 garantiza que $S(T_{2,L}) = B[1 \ldots \text{len} - k]$ y $S(T_{2,R}) = B[\text{len} - k + 1 \ldots \text{len}]$. Por el Lema 2, $\texttt{merge}(T_{2,R}, T_{2,L})$ produce exactamente la permutación deseada.

III-D. Traza de Verificación con el Caso de Ejemplo

En la Tabla II se verifica la ejecución paso a paso de la estructura sobre el caso de prueba oficial ($n = 10, q = 15$), confirmando que el estado interno y las salidas para las 10 consultas `M` coinciden con la especificación.

IV. Análisis de Complejidad

IV-A. Altura Esperada del Treap Implícito

Sea $N = |a|$ la cantidad de elementos activos en el Treap en un instante dado, indexados por su rango *inorder* $1, 2, \ldots, N$. Como las prioridades son variables aleatorias uniformes independientes, el Treap es isomorfo a un BST construido insertando las llaves según el orden decreciente de sus prioridades.

La variable indicadora $X_{i,j} \in \{0, 1\}$ denota si el elemento j es ancestro de i. El elemento j es ancestro de i si y solo si posee la prioridad máxima en el intervalo cerrado $[\min(i, j), \max(i, j)]$:

$$\Pr[X_{i,j} = 1] = \frac{1}{|i - j| + 1} \tag{7}$$

Cuadro II
TRAZA DE EJECUCIÓN SOBRE EL CASO DE PRUEBA OFICIAL

#	Consulta	Estado de la Secuencia a tras la Operación	Salida
0	Inicial	$\{1,2,3,4,5,6,7,8,9,10\}$	–
1	`A 4 8 3`	$\{1,2,3,\mathbf{7},\mathbf{8},\mathbf{9},\mathbf{10},\mathbf{11},9,10\}$	–
2	`M 5 7`	mín$\{8,9,10\}$	**8**
3	`M 7 10`	mín$\{10,11,9,10\}$	**9**
4	`R 2 5`	$\{1,\mathbf{8},\mathbf{7},\mathbf{3},\mathbf{2},9,10,11,9,10\}$	–
5	`M 2 6`	mín$\{8,7,3,2,9\}$	**2**
6	`M 2 3`	mín$\{8,7\}$	**7**
7	`I 3 4`	$\{1,8,7,\mathbf{4},3,2,9,10,11,9,10\}$	–
8	`M 3 4`	mín$\{7,4\}$	**4**
9	`M 5 10`	mín$\{3,2,9,10,11,9\}$	**2**
10	`E 6`	$\{1,8,7,4,3,9,10,11,9,10\}$ ($a_6 = 2$)	–
11	`M 3 5`	mín$\{7,4,3\}$	**3**
12	`M 4 4`	mín$\{4\}$	**4**
13	`O 3 6 3`	$\{1,8,\mathbf{4},\mathbf{3},\mathbf{9},\mathbf{7},10,11,9,10\}$	–
14	`M 5 8`	mín$\{9,7,10,11\}$	**7**
15	`M 7 10`	mín$\{10,11,9,10\}$	**9**

Por linealidad de la esperanza:

$$\begin{aligned}\mathbb{E}[\text{prof}(i)] &= \sum_{j=1, j\neq i}^{N} \mathbb{E}[X_{i,j}] \\ &= (H_i - 1) + (H_{N-i+1} - 1) < 2\ln N = \mathcal{O}(\log N)\end{aligned} \tag{8}$$

donde $H_k = \sum_{m=1}^{k} \frac{1}{m}$ es el k-ésimo número armónico. Mediante cotas de Chernoff, altura$(T) = \mathcal{O}(\log N)$ con probabilidad al menos $1 - N^{-c}$.

IV-B. Complejidad Temporal por Operación y Total

En cada paso recursivo de `split(u, k, L, R)` y `merge(L, R)`, el algoritmo desciende exactamente un nivel en el árbol ejecutando trabajo $\mathcal{O}(1)$ en `push` y `pull`. El tiempo por operación está acotado por $\mathcal{O}(\text{altura}(T)) = \mathcal{O}(\log N)$ en valor esperado.

Cuadro III
COMPLEJIDAD TEMPORAL Y LLAMADAS A PRIMITIVAS POR OPERACIÓN

Operación	`split`	`merge`	Tiempo Esperado
Construcción inicial	0	n	$\mathcal{O}(n \log n)$
`A l r x` (Suma)	2	2	$\mathcal{O}(\log \lvert a \rvert)$
`R l r` (Reversión)	2	2	$\mathcal{O}(\log \lvert a \rvert)$
`O l r k` (Rotación)	3	3	$\mathcal{O}(\log \lvert a \rvert)$
`I k x` (Inserción)	1	2	$\mathcal{O}(\log \lvert a \rvert)$
`E k` (Eliminación)	2	1	$\mathcal{O}(\log \lvert a \rvert)$
`M l r` (Mínimo)	2	2	$\mathcal{O}(\log \lvert a \rvert)$

Dado que $N_{\max} \leq n + q$, la complejidad temporal total del procesamiento en línea es:

$$T(n,q) = \mathcal{O}\big((n+q)\log(n+q)\big) \tag{9}$$

Para $n = q = 5 \cdot 10^5$, esto requiere aproximadamente $4 \cdot 10^7$ operaciones elementales, ejecutadas en menos de 0.45 s en C++ ($< 10\,\%$ del límite de 6.0 s).

IV-C. Complejidad Espacial y Uso de Memoria

En arquitecturas de 64 bits con alineación a 8 bytes, sizeof(Node) $= 48$ bytes:

- Arreglo `tree[1000005]`: $1\,000\,005 \times 48$ B ≈ 45.78 MB.
- Pila `free_list[1000005]`: $1\,000\,005 \times 4$ B ≈ 3.81 MB.
- Pila de recursión: $\mathcal{O}(\log(n+q)) < 1$ KB.

El consumo global es de aproximadamente 49.6 MB ($\mathcal{O}(n+q)$ espacial), muy por debajo del tope de 512 MB.

IV-D. Desempeño por Grupos de Casos de Prueba

La solución propuesta resuelve de forma consistente todos los grupos de evaluación del concurso:

- **Grupo 1 ($n, q \leq 1000$):** El Treap lo resuelve en tiempo despreciable (< 1 ms).
- **Grupos 2 y 3 (Solo `M` / Solo `A` y `M`):** La secuencia es de tamaño estático; el Treap iguala la cota $\mathcal{O}(\log n)$ de un *Segment Tree*.
- **Grupo 4 (Solo `O`, `I`, `E`, `M`) y Grupo 5 (Sin restricciones):** El Treap Implícito absorbe inserciones, eliminaciones, rotaciones y reversiones sin penalización estructural, garantizando puntaje completo (12/12 puntos).

V. CONCLUSIONES

Se diseñó e implementó un BST implícito basado en un *Treap Implícito* con propagación perezosa y memoria preasignada en un *pool* estático. La combinación de tamaños de subárbol como llaves implícitas, prioridades aleatorias de *Max-Heap* y etiquetas diferidas conmutativas permite ejecutar actualizaciones de rango, mutaciones de topología y consultas de mínimo en tiempo esperado $\mathcal{O}(\log n)$ y espacio $\mathcal{O}(n+q)$, satisfaciendo rigurosamente las restricciones operativas.

REFERENCIAS

[1] R. Seidel and C. R. Aragon, "Randomized search trees," *Algorithmica*, vol. 16, no. 4/5, pp. 464–497, 1996.

[2] D. D. Sleator and R. E. Tarjan, "Self-adjusting binary search trees," *Journal of the ACM (JACM)*, vol. 32, no. 3, pp. 652–686, 1985.

[3] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, *Introduction to Algorithms*, 4th ed. Cambridge, MA, USA: MIT Press, 2022.

[4] E. D. Demaine, "6.851: Advanced Data Structures," Massachusetts Institute of Technology (MIT), OpenCourseWare, 2021.